Slice

A **slice** is one of the most important and powerful data structures in **Go**. To understand slices deeply, you must understand **memory layout, slice header, relation with arrays, capacity growth, slicing rules, and internal runtime behavior**.

I'll explain step-by-step from **basic → runtime internals → advanced behavior**.

---

1. What is a Slice in Go?

A **slice is a dynamically-sized view over an underlying array**.

It does **not store elements directly** like arrays.
Instead it contains a **reference to an array**.

Example:

```
arr := [5]int{10, 20, 30, 40, 50}


s := arr[1:4]


fmt.Println(s) // [20 30 40]
```

Here

Array: [10 20 30 40 50]

Index: 0 1 2 3 4

Slice s = arr[1:4]

s -> [20 30 40]

---

2. Slice Internal Structure (Slice Header)

Internally a slice is a **struct with 3 fields**.

```
type slice struct {
   pointer *T
   length  int
   capacity int
}
```

Example:

```
s := []int{10,20,30,40}
```

Slice Header:

pointer → address of first element

length → number of elements

capacity → maximum elements before reallocation

Visual:

```
Slice Header
+-------------------+
| ptr -> array[0]   |
| len = 4           |
| cap = 4           |
+-------------------+


Underlying Array
```

```
+----+----+----+----+
|10  |20  |30  |40  |
+----+----+----+----+
```

---

3. Creating Slices

Method 1 — Slice Literal

s := []int{1,2,3,4}

Memory:

slice header → array created automatically

Method 2 — From Array

arr := [5]int{1,2,3,4,5}

s := arr[1:4]

Result

s = [2 3 4]

len = 3

cap = 4

Why cap=4?

Because capacity is counted from **slice start to end of array**.

array: [1 2 3 4 5]

```
    0 1 2 3 4
```

s := arr[1:4]

start = index 1

capacity = 5 - 1 = 4

Method 3 — Using make()

s := make([]int, 3, 5)

Meaning

length = 3

capacity = 5

Memory

Slice Header

len=3 cap=5

Underlying Array

```
+----+----+----+----+----+
| 0  | 0  | 0  |    |    |
+----+----+----+----+----+
```

---

4. Length vs Capacity

Example:

s := make([]int, 3, 5)

fmt.Println(len(s)) // 3

fmt.Println(cap(s)) // 5

Definitions:

length → number of elements currently in slice

capacity → max elements before new allocation

---

5. Slice Expression

Syntax:

slice[start:end]

Rule:

start ≤ end ≤ capacity

Example:

arr := [6]int{10,20,30,40,50,60}

s := arr[2:5]

Result

s = [30 40 50]

len = 3

cap = 4

Why cap=4?

array size = 6

start index = 2

capacity = 6 - 2 = 4

---

6. Slice Shares Memory (Important)

Slices **share the same underlying array**.

Example:

```
arr := [5]int{1,2,3,4,5}


s1 := arr[1:4]

s2 := arr[2:5]


s1[1] = 100


fmt.Println(arr)

fmt.Println(s2)
```

Output

```
[1 2 100 4 5]

[100 4 5]
```

Memory:

Array

```
[1 2 3 4 5]


s1 -> [2 3 4]

s2 ->   [3 4 5]
```

Change s1[1]

→ array changes

→ s2 also affected

---

7. Append Function

Append adds elements to slice.

```
s := []int{1,2,3}


s = append(s,4)


fmt.Println(s)
```

Output

[1 2 3 4]

---

8. What Happens When Capacity is Full?

Example:

```
s := make([]int,3,3)


s = append(s,4)
```

Since capacity is full:

runtime creates NEW ARRAY

copies elements

adds new element

Example memory change:

Before append

Slice

len=3 cap=3

Array

[1 2 3]

After append

Slice

len=4 cap=6 (runtime grows capacity)

New Array

[1 2 3 4 _ _]

---

9. Capacity Growth Strategy

Go runtime usually grows capacity like:

small slices → double capacity

large slices → ~1.25x growth

Example:

cap=1 → 2

cap=2 → 4

cap=4 → 8

cap=8 → 16

---

10. Copy Function

To copy slices safely.

```
a := []int{1,2,3}

b := make([]int,3)


copy(b,a)


fmt.Println(b)
```

Output

[1 2 3]

Now arrays are **separate**.

---

11. Nil Slice

Declaration:

```
var s []int
```

Properties:

s == nil

len = 0

cap = 0

Example:

```
var s []int
```

```
fmt.Println(s == nil) // true
```

But append works:

```
s = append(s,10)
```

---

12. Empty Slice vs Nil Slice

Nil slice

```
var s []int
```

Empty slice

```
s := []int{}
```

Difference:

nil slice

ptr = nil

len = 0

cap = 0

empty slice

ptr ≠ nil

len = 0

cap = 0

---

13. Full Slice Expression

Advanced syntax:

```
slice[start:end:max]
```

Example:

```
arr := [6]int{1,2,3,4,5,6}


s := arr[1:3:4]
```

Result

s = [2 3]

len = 2

cap = 3

Formula:

cap = max - start

Purpose:

Control **capacity** to prevent append affecting original array.

---

14. Passing Slices to Functions

Slices are passed **by value but reference inside**.

Example:

```
func modify(s []int) {
   s[0] = 100
}


func main() {
   a := []int{1,2,3}
```

```
    modify(a)
    fmt.Println(a)
}
```

Output

[100 2 3]

Because slice header copied but **points to same array**.

---

15. Slice Memory Layout

Complete structure:

```
Slice Variable
+-----------------------+
| pointer → array[0]    |
| len = 3               |
| cap = 5               |
+-----------------------+

Underlying Array
+----+----+----+----+----+
|10  |20  |30  |    |    |
+----+----+----+----+----+
```

---

16. Slice vs Array

Feature	Array	Slice
Size	Fixed	Dynamic
Memory	Inline	Reference
Type	[5]int	[]int
Resizable	❌	✅
Passed to function	Copy entire array	Header copy

---

17. Common Slice Pitfall

Memory leak example:

```
big := make([]byte, 1000000)


small := big[:10]
```

Now **1MB array stays in memory** because slice references it.

Solution:

```
small := append([]byte(nil), big[:10]...)
```

---

18. Slice Operations Summary

Create

```
s := []int{1,2,3}
s := make([]int,5)
```

Slice

```
s[1:4]
```

Append

s = append(s,5)

Copy

copy(dest, src)

Length

len(s)

Capacity

cap(s)

---

19. Real Runtime Slice Representation

In Go runtime (runtime/slice.go)

```
type slice struct {
   array unsafe.Pointer
   len   int
   cap   int
}
```

---

20. Key Concept to Remember

Slice = **descriptor of an array**

slice

↓

slice header

↓

pointer → underlying array

Slice escape analysis (stack vs heap)

Understanding slice escape analysis is very important for writing high-performance Go programs because it determines whether memory is allocated on the stack or the heap.

Go compiler automatically decides this using escape analysis.

---

1. What is Escape Analysis?

Escape analysis is a compiler optimization that determines whether a variable can stay on the stack or must move to the heap.

Rule:

If data does NOT escape the function → stack allocation

If data escapes the function → heap allocation

Why?

Stack memory → fast, automatically freed

Heap memory → slower, needs garbage collection

---

2. Stack vs Heap in Go

Stack Allocation

Characteristics

- **very fast**
- **automatically freed when function returns**

- no garbage collection needed

Example:

```
func main() {
   x := 10
   fmt.Println(x)
}
```

Memory

Stack Frame (main)

---

x = 10

Heap Allocation

Characteristics

- slower
- managed by garbage collector
- used when data outlives function scope

Example:

```
func create() *int {
   x := 10
   return &x
}
```

Here x escapes the function.

create() returns pointer to x

So Go moves x to heap.

---

3. Escape Analysis With Slices

Slices are interesting because slice header and underlying array behave differently.

Slice internally:

```
type slice struct {
   ptr *array
   len int
   cap int
}
```

Two things can escape:

1. slice header
2. underlying array

---

4. Example 1 — Slice on Stack

```
func main() {
   s := []int{1,2,3}
   fmt.Println(s)
}
```

Memory

Stack

slice header

ptr → array

len

cap

array [1 2 3]

Nothing escapes.

So both stay on stack.

---

5. Example 2 — Slice Returned From Function

```
func create() []int {
   s := []int{1,2,3}
   return s
}
```

Here slice escapes the function.

Memory layout:

Heap

array [1 2 3]

Stack

slice header

Why?

Because returned slice must survive after function exits.

---

6. Example 3 — Large Slice Allocation

```
func main() {
   s := make([]int, 1000000)
   fmt.Println(len(s))
}
```

Even though slice does not escape, Go may allocate large arrays on heap.

Reason:

stack has limited size

large allocations go to heap

---

7. Example 4 — Slice Passed to Another Function

```
func process(s []int) {
   fmt.Println(s)
}


func main() {
   s := []int{1,2,3}
   process(s)
}
```

Here slice does NOT escape.

Memory

Stack

slice header copied

same underlying array

Still stack allocation.

---

8. Example 5 — Slice Stored in Interface (Escape)

```
func main() {
   s := []int{1,2,3}
   var i interface{} = s
   fmt.Println(i)
}
```

Why escape?

Because interface may outlive function.

So compiler moves slice to heap.

---

9. Example 6 — Returning Pointer to Slice Element

```
func get() *int {
   s := []int{1,2,3}
   return &s[0]
}
```

Now slice must survive after function returns.

Memory

Heap

array [1 2 3]

Because pointer references it.

---

10. How to Check Escape Analysis

Go compiler can show escape results.

Command:

```
go build -gcflags="-m"
```

Example output:

```
./main.go:10:6: moved to heap: s
./main.go:11:13: s escapes to heap
```

Meaning:

compiler decided heap allocation

---

11. Example with Escape Analysis Output

Code

```
func create() []int {
   s := []int{1,2,3}
   return s
}
```

Compile:

```
go build -gcflags="-m" main.go
```

Output

```
s escapes to heap
```

---

12. Slice Append Escape Behavior

Example:

```
func build() []int {
   s := []int{}
   for i:=0;i<10;i++{
      s = append(s,i)
   }
   return s
}
```

Here underlying array likely allocated on heap.

Reason:

append grows dynamically

lifetime extends outside function

---

13. Common Escape Situations With Slices

Slices escape when:

1. returned from function

2. stored in interface

3. referenced by pointer

4. used in goroutines

5. captured by closures

6. large memory allocation

---

14. Closure Escape Example

```
func main() {
   s := []int{1,2,3}


   go func() {
      fmt.Println(s)
   }()
}
```

Now slice escapes.

Because goroutine runs after function returns.

Memory:

Heap

array [1 2 3]

---

15. Performance Impact

Stack

- **no GC**
- **extremely fast**

Heap

- **GC required**
- **slower allocation**
- **more memory pressure**

So avoiding unnecessary escapes improves performance.

---

16. Real Production Example

Bad:

```
func build() []int {
   s := []int{}
   for i := 0; i < 1000; i++ {
      s = append(s, i)
   }
   return s
}
```

Better:

```
func build() []int {
   s := make([]int,0,1000)
   for i := 0; i < 1000; i++ {
      s = append(s, i)
   }
   return s
}
```

Why better?

- fewer reallocations
- predictable memory usage

---

17. Key Concept

slice header → small (3 words)

underlying array → actual memory

Escape analysis mainly affects underlying array.

---

18. Visual Summary

Stack Allocation

Stack

----

slice header

pointer → array

len

cap

array [1 2 3]

Heap Allocation

Stack

slice header

Heap

array [1 2 3]

---

19. Golden Rule

If slice data needs to live beyond function scope → heap.

Otherwise stack.

In Go, slices are commonly used to implement CRUD operations (Create, Read, Update, Delete) when you are managing in-memory collections (like a list of users, products, etc.).

Let's understand CRUD using slices step-by-step with a realistic example.

---

1. Example Data Structure

Suppose we store users in a slice.

```
type User struct {
    ID   int
    Name string
}
```

Slice to store users:

```
var users []User
```

---

2. CREATE Operation (Insert)

Add a new element using append().

```
func createUser(id int, name string) {
    user := User{id, name}
    users = append(users, user)
}
```

Example:

```
createUser(1, "Alice")
```

```
createUser(2, "Bob")

createUser(3, "Charlie")
```

Memory after insert:

```
users slice

+--------------------------+

| {1 Alice} {2 Bob} {3 Charlie} |

+--------------------------+
```

---

3. READ Operation

Read elements from the slice.

Read All Users

```
func getUsers() {
	for _, u := range users {
		fmt.Println(u.ID, u.Name)
	}
}
```

Output

```
1 Alice

2 Bob

3 Charlie
```

Read by ID

```
func getUserByID(id int) *User {
```

```
	for i := range users {
		if users[i].ID == id {
			return &users[i]
		}
	}
	return nil
}
```

Usage:

```
u := getUserByID(2)

if u != nil {
	fmt.Println(u.Name)
}
```

Output

Bob

---

4. UPDATE Operation

Modify an existing element.

```
func updateUser(id int, newName string) bool {
	for i := range users {
		if users[i].ID == id {
			users[i].Name = newName
			return true
```

```
            }
        }
        return false
}
```

Example:

updateUser(2, "Robert")

Result:

{1 Alice}

{2 Robert}

{3 Charlie}

---

5. DELETE Operation

To remove an element from a slice:

append(slice[:i], slice[i+1:]...)

Implementation:

```
func deleteUser(id int) bool {
        for i := range users {
            if users[i].ID == id {
                users = append(users[:i], users[i+1:]...)
                return true
            }
        }
        return false
```

}

Example:

deleteUser(2)

Result:

{1 Alice}

{3 Charlie}

Memory transformation:

Before:

[Alice | Bob | Charlie]

After delete:

append(users[:1], users[2:]...)

[Alice | Charlie]

---

6. Complete Working Example

```
package main

import "fmt"

type User struct {
	ID   int
	Name string
}
```

```go
var users []User

func createUser(id int, name string) {
	users = append(users, User{id, name})
}

func getUsers() {
	for _, u := range users {
		fmt.Println(u.ID, u.Name)
	}
}

func updateUser(id int, newName string) bool {
	for i := range users {
		if users[i].ID == id {
			users[i].Name = newName
			return true
		}
	}
	return false
}
```

```go
func deleteUser(id int) bool {
	for i := range users {
		if users[i].ID == id {
			users = append(users[:i], users[i+1:]...)
			return true
		}
	}
	return false
}

func main() {

	createUser(1, "Alice")
	createUser(2, "Bob")
	createUser(3, "Charlie")

	fmt.Println("All Users")
	getUsers()

	fmt.Println("Update User")
	updateUser(2, "Robert")
	getUsers()
```

```
    fmt.Println("Delete User")
    deleteUser(1)
    getUsers()
}
```

Output

All Users

1 Alice

2 Bob

3 Charlie

Update User

1 Alice

2 Robert

3 Charlie

Delete User

2 Robert

3 Charlie

---

7. Performance Considerations

Operation	Time Complexity
Create (append)	O(1) amortized

Operation	Time Complexity
Read	O(n)
Update	O(n)
Delete	O(n)

Reason:

Slice is backed by array

Deleting requires shifting elements

---

8. Production Optimization Trick

If order does not matter, deletion can be O(1).

Example:

```
func deleteFast(i int) {
    users[i] = users[len(users)-1]
    users = users[:len(users)-1]
}
```

Before:

[A B C D]

delete B

After:

[A D C]

No shifting required.

---

9. CRUD Memory Visualization

Create

append → new element added

Read

iterate slice

Update

modify element

Delete

append(slice[:i], slice[i+1:]...)